

Dabai_DCW2 Camera Introduction

Dabai_DCW2 Camera Introduction

1. Course Content
2. Camera Introduction
3. Start Camera Node
 - 3.1 Subscribe to Color Image
 - 3.2 Subscribe to Depth Image
 - 3.3 Subscribe to Infrared Image
 - 3.4 Subscribe to Image Point Cloud

1. Course Content

[!IMPORTANT]

This course section is only for users with the Dabai_DCW2 camera package to view and reference

- Run example programs, use Orbbec's depth camera - Dabai_DCW2, to view color images, depth images, infrared images, and point clouds published by camera nodes.

2. Camera Introduction

DaBai DCW2/DW2 depth camera obtains object depth images based on binocular structured light 3D imaging technology, while using a color camera to capture object color images. It is suitable for intelligent products that perform 3D object scanning at distances of 0.2m-5m, and can achieve object depth data measurement within the measurement range.



IR: Infrared camera

LDP: Laser dot matrix projector

LDM: Laser drive module

RGB: Color camera

Features of DaBai DCW2:

- Low reflectivity objects: Can effectively recognize objects with 5% reflectivity
- Anti-interference: Comprehensively improved electromagnetic compatibility and anti-static capabilities
- Balanced field of view: H-FOV: 91° V-FOV: 62°

- Energy level switching: Provides two energy level modes for customer adaptation
- Depth image: Supports up to 640*400 depth resolution
- Working distance: 0.15m-5m
- Depth accuracy: <1%@1m
- Interface: USB2.0 Type C

3. Start Camera Node

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

- Enter command in terminal

```
ros2 launch m3_bringup bringup_camera.launch.py
```

Successfully driven camera screen is as follows

```
root@raspberrypi:~# ros2 launch orbbeec_camera dabai_dcw2.launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2025-06-13-02-14-59-847729-raspberrypi-359
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [component_container-1]: process started with pid [371]
[component_container-1] [INFO] [1749780900.465207801] [camera.camera_container]: Load Library: /root/yahboomcar_ws/install/orbbeeec_camera/lib/liborbbeeec_camera.so
[component_container-1] [INFO] [1749780901.492742908] [camera.camera_container]: Found class: rclcpp_components::NodeFactoryTemplate<orbbeeec_camera::OBCameraNodeDriver>
[component_container-1] [INFO] [1749780901.492910480] [camera.camera_container]: Instantiate class: rclcpp_components::NodeFactoryTemplate<orbbeeec_camera::OBCameraNodeDriver>
[component_container-1] [06/13 02:15:01.515757][info][371][Context.cpp:68] Context created with config: /root/yahboomcar_ws/install/orbbeeec_camera/share/orbbeeec_camera/config/OrbbeeecSDKConfig_v1.0.xml
[component_container-1] [06/13 02:15:01.515770][info][371][Context.cpp:73] Work directory=/root, SDK version=v1.10.15-20241012-bb4e350
[component_container-1] [06/13 02:15:01.515781][info][371][LinuxPal.cpp:32] createObPal: create LinuxPal!
[component_container-1] [06/13 02:15:01.523328][warning][371][OpenNIDeviceInfo.cpp:186] New openni device matched.
[component_container-1] [06/13 02:15:01.529380][info][371][LinuxPal.cpp:166] Create PollingDeviceWatcher!
[component_container-1] [06/13 02:15:01.529740][info][371][DeviceManager.cpp:15] Current found device(s): (1)
[component_container-1] [06/13 02:15:01.529771][info][371][DeviceManager.cpp:24] - Name: DaBai DCW2, PID: 0x06a0, SN/ID: AUIL93D005A, Connection: USB2.0
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/camera/camera' in container '/camera/camera_container'
[component_container-1] [INFO] [1749780901.635874317] [camera.camera]: Connecting to the default device
[component_container-1] [INFO] [1749780901.841942792] [camera.camera]: Select device cost 206 ms
[component_container-1] [INFO] [1749780901.842019717] [camera.camera]: Try to connect device via USB2.0
[component_container-1] [INFO] [1749780901.856877491] [camera.camera]: OBCameraNode: use_intra_process: OFF
[component_container-1] [INFO] [1749780901.861393872] [camera.camera]: current time domain: system
```

- If the camera cannot be started, you need to check if the camera connection to the mainboard/Hub expansion board is loose. You can use ros2 node tools to view which topics the camera node has published and which services are provided. Enter in terminal,

```
ros2 node info /camera/camera
```

Published topics are as shown in the figure below,

```

Publishers:
/camera/color/camera_info: sensor_msgs/msg/CameraInfo
/camera/color/image_raw: sensor_msgs/msg/Image
/camera/color/image_raw/compressed: sensor_msgs/msg/CompressedImage
/camera/color/image_raw/compressedDepth: sensor_msgs/msg/CompressedImage
/camera/color/image_raw/theora: theora_image_transport/msg/Packet
/camera/depth/camera_info: sensor_msgs/msg/CameraInfo
/camera/depth/image_raw: sensor_msgs/msg/Image
/camera/depth/image_raw/compressed: sensor_msgs/msg/CompressedImage
/camera/depth/image_raw/compressedDepth: sensor_msgs/msg/CompressedImage
/camera/depth/image_raw/theora: theora_image_transport/msg/Packet
/camera/depth/points: sensor_msgs/msg/PointCloud2
/camera/depth_filter_status: std_msgs/msg/String
/camera/depth_registered/points: sensor_msgs/msg/PointCloud2
/camera/depth_to_color: orbbec_camera_msgs/msg/Extrinsics
/camera/depth_to_ir: orbbec_camera_msgs/msg/Extrinsics
/camera/ir/camera_info: sensor_msgs/msg/CameraInfo
/camera/ir/image_raw: sensor_msgs/msg/Image
/camera/ir/image_raw/compressed: sensor_msgs/msg/CompressedImage
/camera/ir/image_raw/compressedDepth: sensor_msgs/msg/CompressedImage
/camera/ir/image_raw/theora: theora_image_transport/msg/Packet
/diagnostics: diagnostic_msgs/msg/DiagnosticArray
/parameter_events: rcl_interfaces/msg/ParameterEvent
/rosout: rcl_interfaces/msg/Log
/tf: tf2_msgs/msg/TFMessage
/tf_static: tf2_msgs/msg/TFMessage

```

Provided services are as shown in the figure below,

```

Service Servers:
/camera/camera/describe_parameters: rcl_interfaces/srv/DescribeParameters
/camera/camera/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
/camera/camera/get_parameters: rcl_interfaces/srv/GetParameters
/camera/camera/list_parameters: rcl_interfaces/srv/ListParameters
/camera/camera/set_parameters: rcl_interfaces/srv/SetParameters
/camera/camera/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
/camera/get_auto_white_balance: orbbec_camera_msgs/srv/GetInt32
/camera/get_color_exposure: orbbec_camera_msgs/srv/GetInt32
/camera/get_color_gain: orbbec_camera_msgs/srv/GetInt32
/camera/get_depth_exposure: orbbec_camera_msgs/srv/GetInt32
/camera/get_depth_gain: orbbec_camera_msgs/srv/GetInt32
/camera/get_device_info: orbbec_camera_msgs/srv/GetDeviceInfo
/camera/get_ir_exposure: orbbec_camera_msgs/srv/GetInt32
/camera/get_ir_gain: orbbec_camera_msgs/srv/GetInt32
/camera/get_ldp_measure_distance: orbbec_camera_msgs/srv/GetInt32
/camera/get_ldp_status: orbbec_camera_msgs/srv/GetBool
/camera/get_sdk_version: orbbec_camera_msgs/srv/GetString
/camera/get_white_balance: orbbec_camera_msgs/srv/GetInt32
/camera/reboot_device: std_srvs/srv/Empty
/camera/save_images: std_srvs/srv/Empty
/camera/save_point_cloud: std_srvs/srv/Empty
/camera/set_auto_white_balance: std_srvs/srv/SetBool
/camera/set_color_auto_exposure: std_srvs/srv/SetBool
/camera/set_color_exposure: orbbec_camera_msgs/srv/SetInt32

```

3.1 Subscribe to Color Image

According to the queried node information, the following color image topics can be found;

/camera/color/image_raw: Color image topic

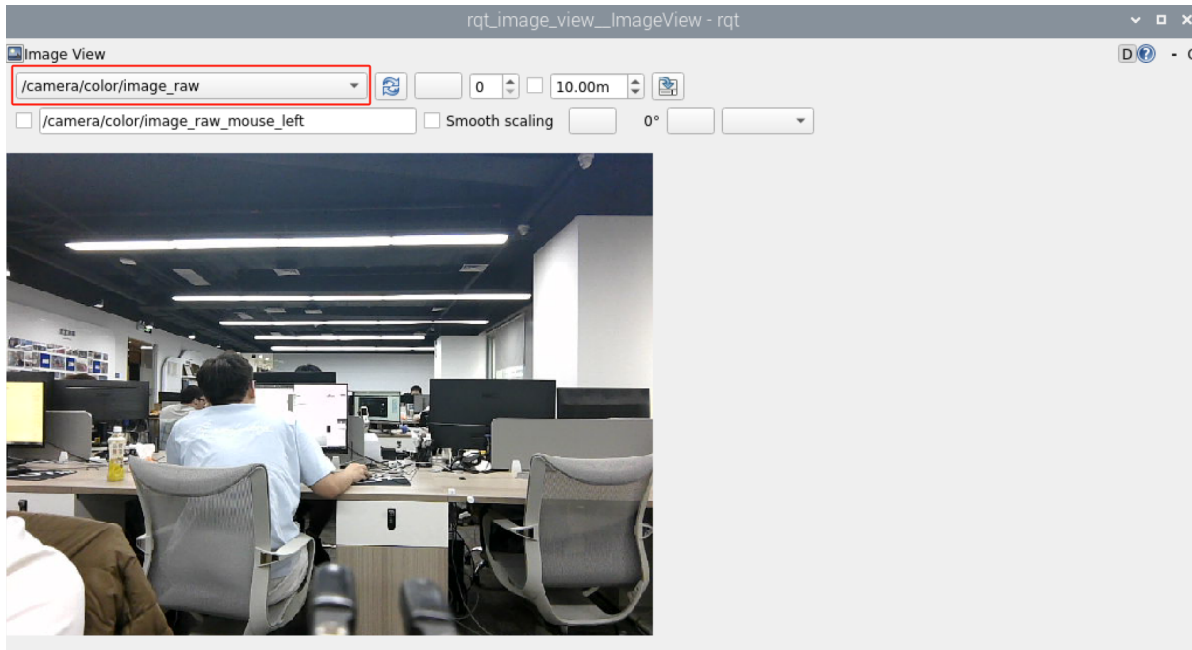
/camera/color/image_raw/compressed: Compressed color image topic

/camera/color/image_raw/compressedDepth: Depth compressed color image topic

/camera/color/image_raw/theora: Color image topic compressed using theora encoder

After the camera is started, you can enter the following command in the terminal to start the `rqt_images_view` tool, and view images according to the selected image topic. Enter in terminal,

```
ros2 run rqt_image_view rqt_image_view
```



As shown in the red box in the figure above, the color image topic `/camera/color/image_raw` is selected for display.

3.2 Subscribe to Depth Image

According to the queried node information, the following depth image topics can be found:

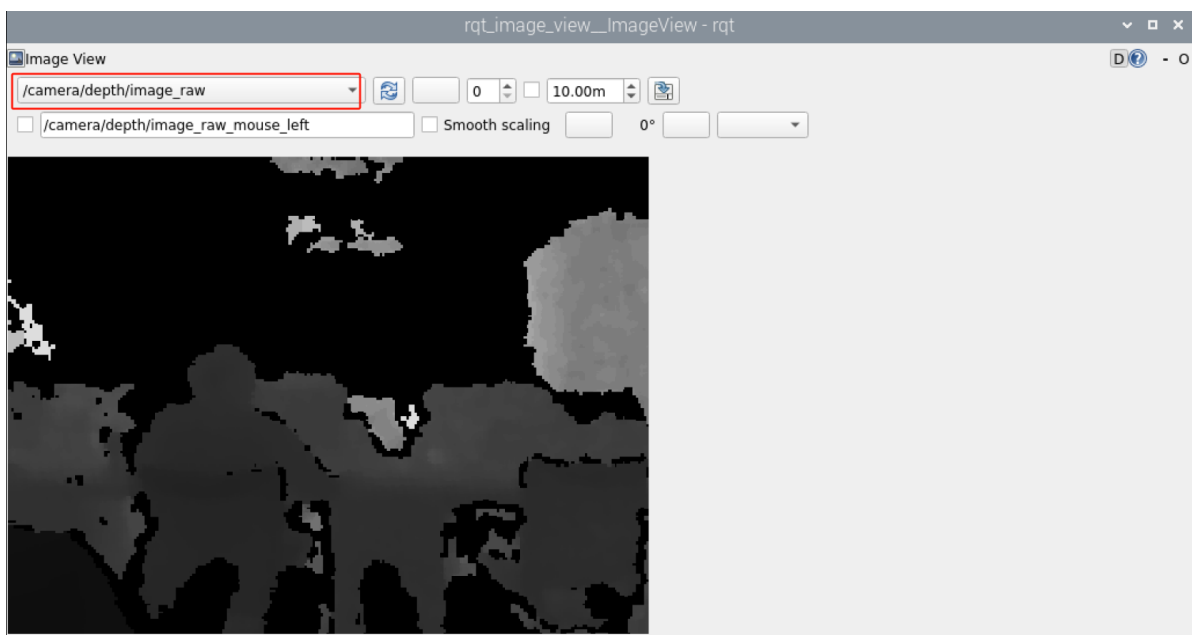
`/camera/depth/image_raw`: Depth image topic

`/camera/depth/image_raw/compressed`: Compressed depth image topic

`/camera/depth/image_raw/compressedDepth`: Depth compressed depth image topic

`/camera/depth/image_raw/theora`: Depth image topic compressed using theora encoder

Similarly, we can enter the command mentioned above `ros2 run rqt_image_view rqt_image_view` in the terminal to start the `rqt_images_view` tool, and select the depth image topic to display depth images,



As shown in the figure above, the depth image of the /camera/depth/image_raw depth image topic is displayed.

3.3 Subscribe to Infrared Image

According to the queried node information, the following infrared image topics can be found:

/camera/ir/image_raw: Infrared image topic

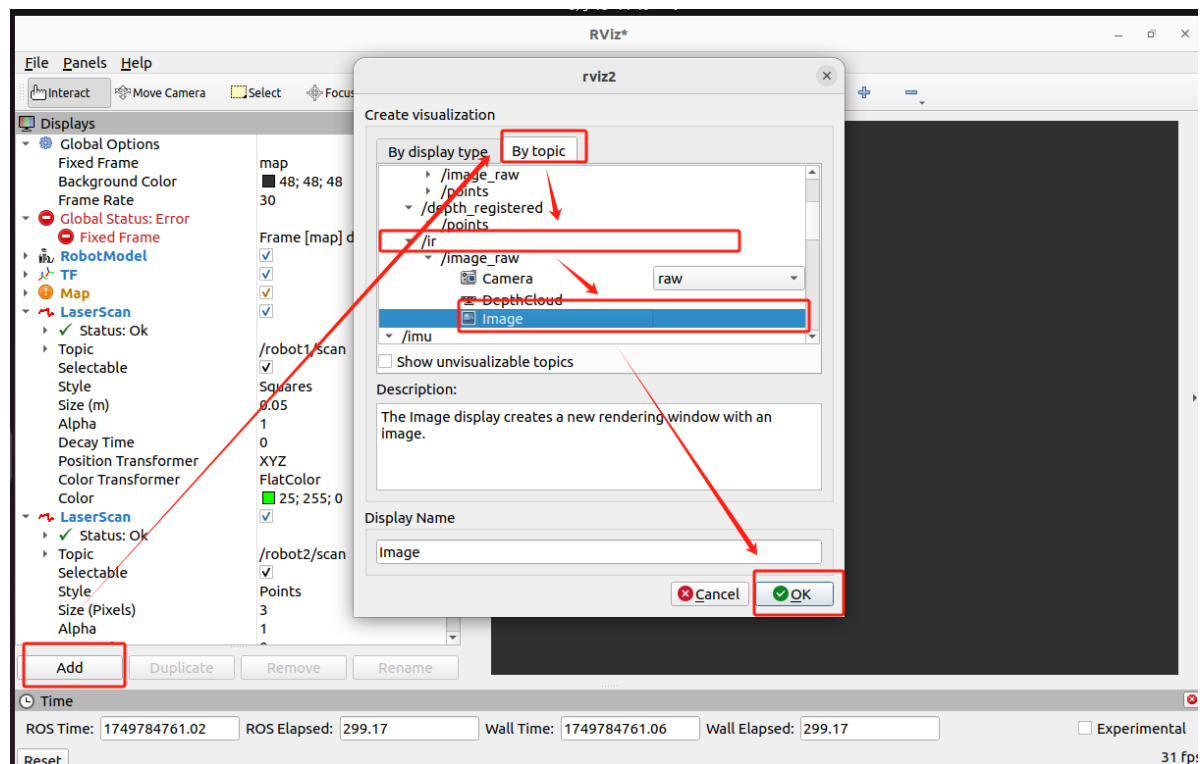
/camera/ir/image_raw/compressed: Compressed infrared image topic

/camera/ir/image_raw/compressedDepth: Depth compressed infrared image topic

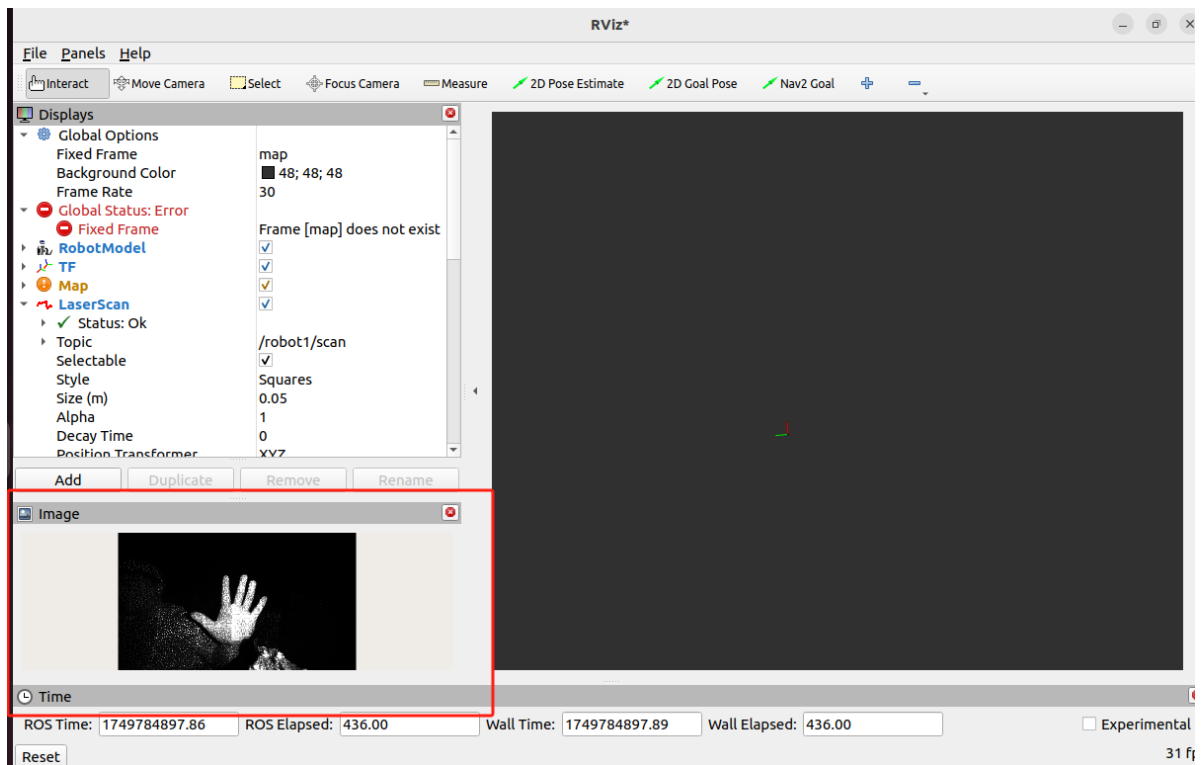
/camera/ir/image_raw/theora: Infrared image topic compressed using theora encoder

rqt_image_view cannot display infrared images, we need to use the rviz tool to display them. Use the virtual machine and the car for distributed communication, start rviz on the virtual machine side to select plugins to display infrared images. Before this, you need to set up distributed communication between the virtual machine and the car. For detailed setup steps, please refer to the content in [0. Robot Configuration and Operation Guide] - [Car Virtual Machine Distributed Communication Setup] in this product tutorial. Open a terminal in the virtual machine and enter the following command to start rviz,

```
rviz2
```



Follow the arrow instructions in the figure above, and finally click [ok] to complete the selection. As shown in the figure below, the infrared image is successfully displayed



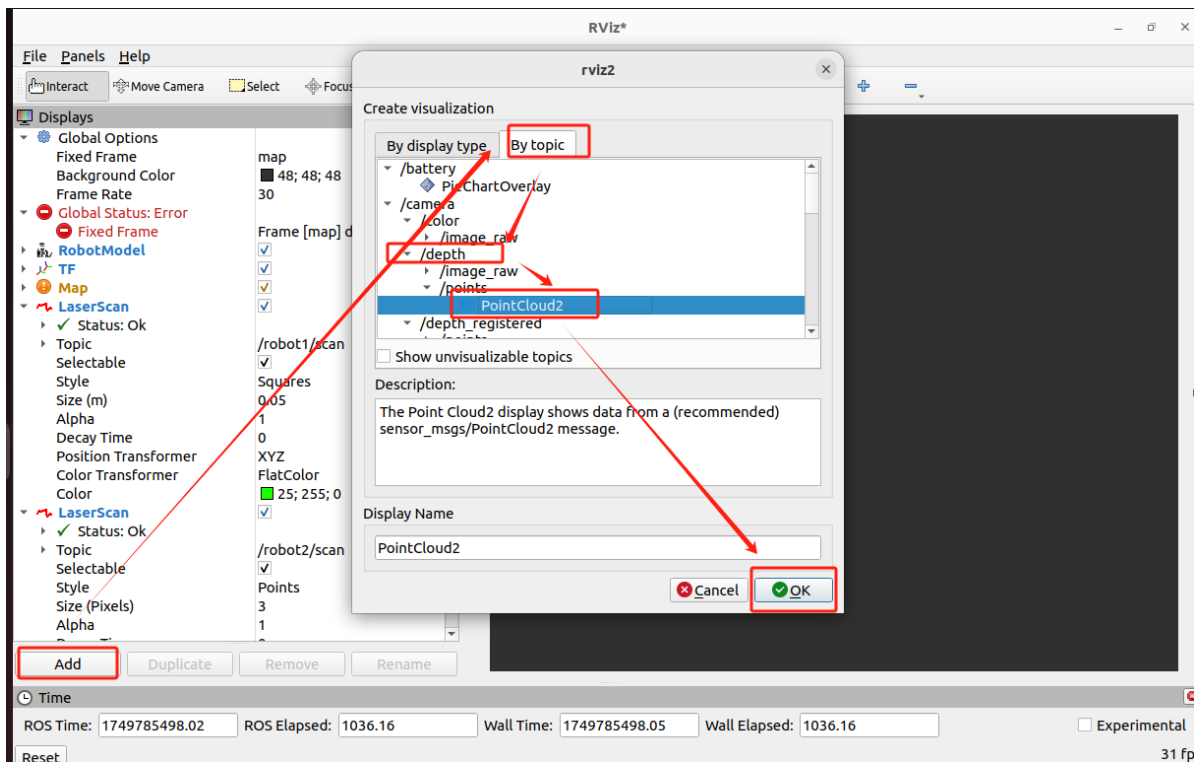
3.4 Subscribe to Image Point Cloud

According to the queried node information, it can be found that the camera node publishes image point clouds

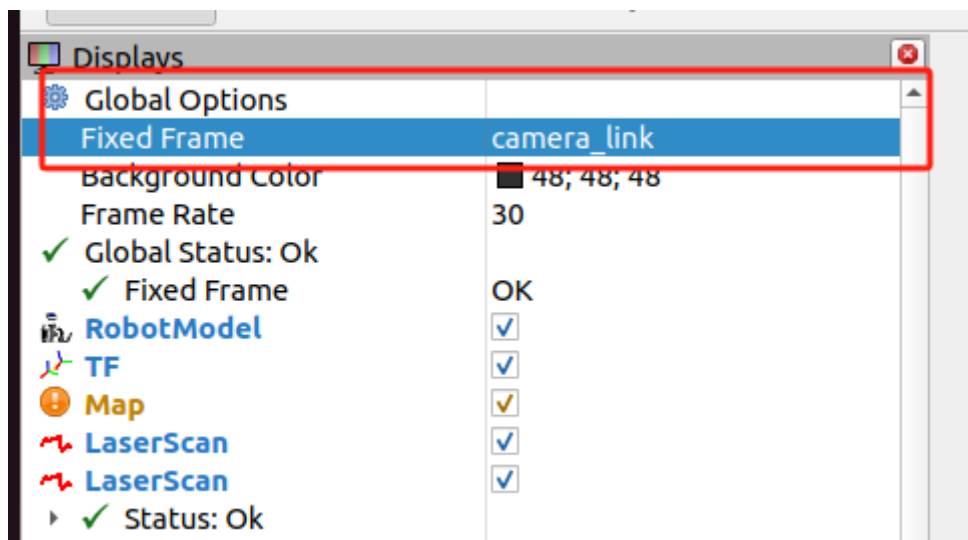
/camera/depth/points: 3D depth point cloud information

/camera/depth_registered/points: Registered 3D point cloud topic (depth data (Depth) has been **aligned (registered)** with color image (RGB), ensuring each depth point corresponds to the correct color)

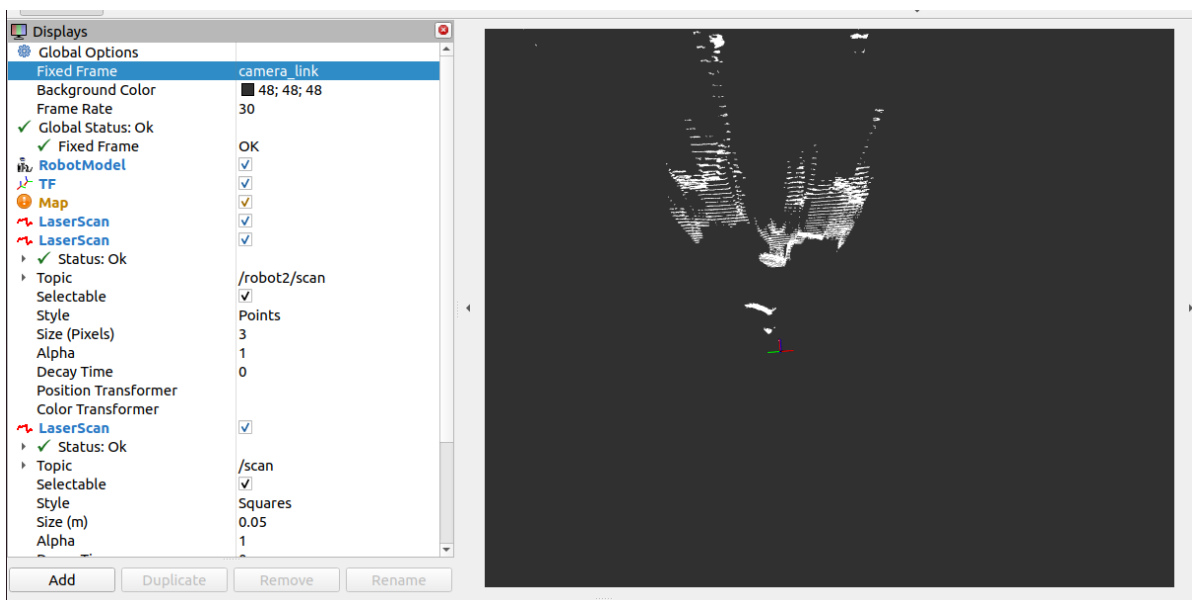
Use the rviz tool to display point cloud information. Similarly, you need to set up distributed communication between the virtual machine and the car. We enter `rviz2` in the virtual machine terminal to start rviz, then make selections according to the following steps,



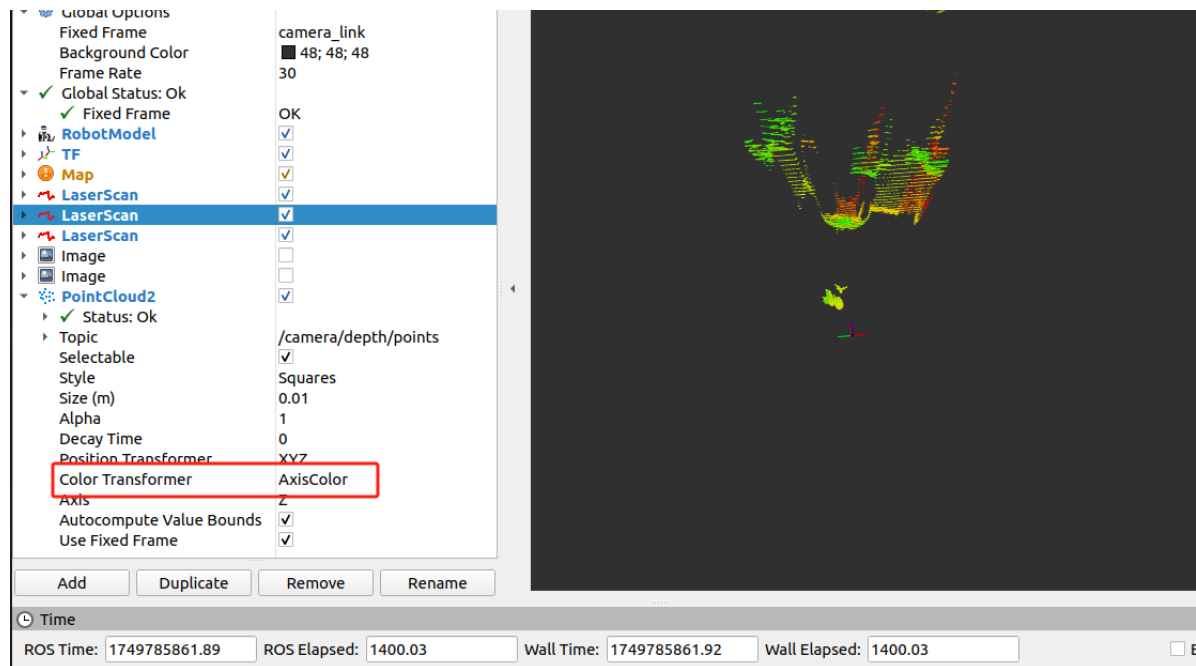
After selection is complete, we also need to set the coordinate system to camera_link, as shown in the figure below,



As shown in the figure below, the /camera/depth/points topic point cloud is displayed,



We can also modify the display settings of point clouds,



In [topic], select /camera/depth_registered/points to display the point cloud after aligning depth and color images; in [Color Transformer], select RGB8 to display RGB point clouds, as shown in the figure below

